

PROYECTO FINAL – RED SOCIAL ACADÉMICA

Ericka Daniela Caldera Gómez

Carné: 9941 – 24 – 8190

1. Introducción

El presente proyecto consiste en el desarrollo de una Red Social Académica utilizando estructuras de datos avanzadas en C++ y un benchmark comparativo utilizando estructuras nativas de Java.

El sistema permite gestionar estudiantes, conexiones académicas, proyectos, rankings y realizar comparaciones de rendimiento entre diferentes estructuras de datos.

```
1. Cargar estudiantes desde CSV
2. Registrar estudiante manualmente
3. Buscar estudiante por ID
4. Eliminar estudiante
5. Agregar proyecto
6. Mostrar proyectos
7. Conectar estudiantes
8. Verificar conexiones
9. Mostrar conexiones
10. BFS
11. DFS
12. Mostrar ranking
13. Mostrar estadísticas
14. Mostrar historial
15. Exportar reporte
16. Salir
Seleccione opcion: 13

===== ESTADISTICAS HASH =====
Total estudiantes: 10003
Total colisiones: 9993
Factor carga: 1000.3

===== ESTADISTICAS AVL =====
Altura del arbol: 2
Factor de balance raiz: 0
Cantidad de nodos: 3
```

2. Evidencias de Funcionamiento

Carga masiva desde CSV

```
Nombre: ERICKA DANIELA CALDERA GOMEZ
Carne: 9941 - 24 - 8190

===== RED SOCIAL ACADEMICA =====
1. Cargar estudiantes desde CSV
2. Registrar estudiante manualmente
3. Buscar estudiante por ID
4. Eliminar estudiante
5. Agregar proyecto
6. Mostrar proyectos
7. Conectar estudiantes
8. Verificar conexiones
9. Mostrar conexiones
10. BFS
11. DFS
12. Mostrar ranking
13. Mostrar estadísticas
14. Mostrar historial
15. Exportar reporte
16. Salir
Seleccione opcion: 1
CSV cargado correctamente
```

Se realizó la carga automática de estudiantes desde el archivo estudiantes.csv.

Registro y búsqueda de estudiantes

<pre>===== RED SOCIAL ACADEMICA ===== 1. Cargar estudiantes desde CSV 2. Registrar estudiante manualmente 3. Buscar estudiante por ID 4. Eliminar estudiante 5. Agregar proyecto 6. Mostrar proyectos 7. Conectar estudiantes 8. Verificar conexiones 9. Mostrar conexiones 10. BFS 11. DFS 12. Mostrar ranking 13. Mostrar estadísticas 14. Mostrar historial 15. Exportar reporte 16. Salir Seleccione opcion: 2 ID: 998 Nombre: Brandon Pretzantzin Carrera: Ingeniería en Software Semestre: 5 GPA: 99.9 Skill Score: 800 Acción agregada al historial. Estudiante agregado</pre>	<pre>===== RED SOCIAL ACADEMICA ===== 1. Cargar estudiantes desde CSV 2. Registrar estudiante manualmente 3. Buscar estudiante por ID 4. Eliminar estudiante 5. Agregar proyecto 6. Mostrar proyectos 7. Conectar estudiantes 8. Verificar conexiones 9. Mostrar conexiones 10. BFS 11. DFS 12. Mostrar ranking 13. Mostrar estadísticas 14. Mostrar historial 15. Exportar reporte 16. Salir Seleccione opcion: 3 Ingreso ID: 2001 Nombre: Maria Escobar Mendez Carrera: Ingeniería Administrativa GPA: 87.98</pre>
---	---

Los estudiantes pueden registrarse manualmente y ser localizados mediante su identificador único.

Funcionamiento de la Tabla Hash

```
===== RED SOCIAL ACADEMICA =====
1. Cargar estudiantes desde CSV
2. Registrar estudiante manualmente
3. Buscar estudiante por ID
4. Eliminar estudiante
5. Agregar proyecto
6. Mostrar proyectos
7. Conectar estudiantes
8. Verificar conexiones
9. Mostrar conexiones
10. BFS
11. DFS
12. Mostrar ranking
13. Mostrar estadísticas
14. Mostrar historial
15. Exportar reporte
16. Salir
Seleccione opcion: 3
Ingrese ID: 2001

Nombre: Maria Escobar Mendez
Carrera: Ingenieria Administrativa
GPA: 87.98
```

La Tabla Hash se utiliza para almacenar y recuperar estudiantes mediante su ID de forma eficiente.

Funcionamiento del Árbol AVL

```
===== RED SOCIAL ACADEMICA =====
1. Cargar estudiantes desde CSV
2. Registrar estudiante manualmente
3. Buscar estudiante por ID
4. Eliminar estudiante
5. Agregar proyecto
6. Mostrar proyectos
7. Conectar estudiantes
8. Verificar conexiones
9. Mostrar conexiones
10. BFS
11. DFS
12. Mostrar ranking
13. Mostrar estadísticas
14. Mostrar historial
15. Exportar reporte
16. Salir
Seleccione opcion: 12

===== RANKING ACADEMICO =====
Jose Lopez - Score: 500
Ericka Caldera - Score: 400
Brenda Ramirez - Score: 100
```

El Árbol AVL organiza a los estudiantes según su skill score y permite generar el ranking académico.

Funcionamiento del Grafo

```
===== RED SOCIAL ACADEMICA =====
1. Cargar estudiantes desde CSV
2. Registrar estudiante manualmente
3. Buscar estudiante por ID
4. Eliminar estudiante
5. Agregar proyecto
6. Mostrar proyectos
7. Conectar estudiantes
8. Verificar conexiones
9. Mostrar conexiones
10. BFS
11. DFS
12. Mostrar ranking
13. Mostrar estadísticas
14. Mostrar historial
15. Exportar reporte
16. Salir
Seleccione opcion: 7
ID 1: 1
ID 2: 2
Conexion creada
Accion agregada al historial.

===== RED SOCIAL ACADEMICA =====
1. Cargar estudiantes desde CSV
2. Registrar estudiante manualmente
3. Buscar estudiante por ID
4. Eliminar estudiante
5. Agregar proyecto
6. Mostrar proyectos
7. Conectar estudiantes
8. Verificar conexiones
9. Mostrar conexiones
10. BFS
11. DFS
12. Mostrar ranking
13. Mostrar estadísticas
14. Mostrar historial
15. Exportar reporte
16. Salir
Seleccione opcion: 9
Ingrese ID: 1

Conexiones de 1:
2

===== RED SOCIAL ACADEMICA =====
1. Cargar estudiantes desde CSV
2. Registrar estudiante manualmente
3. Buscar estudiante por ID
4. Eliminar estudiante
5. Agregar proyecto
6. Mostrar proyectos
7. Conectar estudiantes
8. Verificar conexiones
9. Mostrar conexiones
10. BFS
11. DFS
12. Mostrar ranking
13. Mostrar estadísticas
14. Mostrar historial
15. Exportar reporte
16. Salir
Seleccione opcion: 9
Ingrese ID: 2

Conexiones de 2:
1
3
```

El grafo representa las conexiones entre estudiantes y evita conexiones duplicadas.

Recorrido BFS

```
===== RED SOCIAL ACADEMICA =====
1. Cargar estudiantes desde CSV
2. Registrar estudiante manualmente
3. Buscar estudiante por ID
4. Eliminar estudiante
5. Agregar proyecto
6. Mostrar proyectos
7. Conectar estudiantes
8. Verificar conexiones
9. Mostrar conexiones
10. BFS
11. DFS
12. Mostrar ranking
13. Mostrar estadísticas
14. Mostrar historial
15. Exportar reporte
16. Salir
Seleccione opcion: 10
Nodo inicio BFS: 1
BFS: 1 2 3
```

El recorrido BFS explora la red académica por niveles a partir de un estudiante seleccionado.

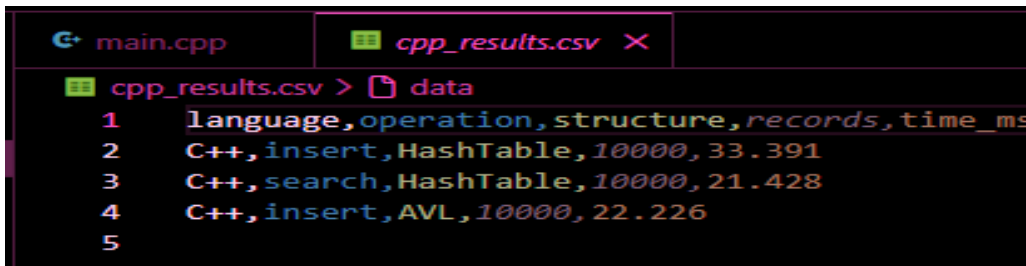
Recorrido DFS

```
===== RED SOCIAL ACADEMICA =====
1. Cargar estudiantes desde CSV
2. Registrar estudiante manualmente
3. Buscar estudiante por ID
4. Eliminar estudiante
5. Agregar proyecto
6. Mostrar proyectos
7. Conectar estudiantes
8. Verificar conexiones
9. Mostrar conexiones
10. BFS
11. DFS
12. Mostrar ranking
13. Mostrar estadísticas
14. Mostrar historial
15. Exportar reporte
16. Salir
Seleccione opcion: 11
Nodo inicio DFS: 2
DFS: 2 1 3
```

El recorrido DFS explora completamente una ruta antes de retroceder.

Generación de cpp_results.csv

```
PS C:\Users\Daniela Caldera\C++ & java\PROYECTO FINAL - Programación III\cpp_system> .\benchmark
Benchmark generado correctamente
PS C:\Users\Daniela Caldera\C++ & java\PROYECTO FINAL - Programación III\cpp_system>
PS C:\Users\Daniela Caldera\C++ & java\PROYECTO FINAL - Programación III\cpp_system> █
```

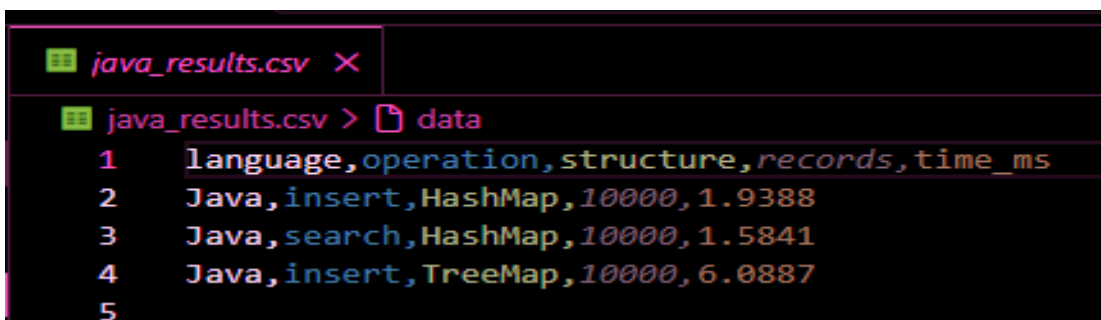


language	operation	structure	records	time_ms
C++	insert	HashTable	10000	33.391
C++	search	HashTable	10000	21.428
C++	insert	AVL	10000	22.226
C++	insert	AVL	10000	22.226
C++	insert	AVL	10000	22.226

El benchmark desarrollado en C++ genera automáticamente el archivo cpp_results.csv.

Generación de java_results.csv

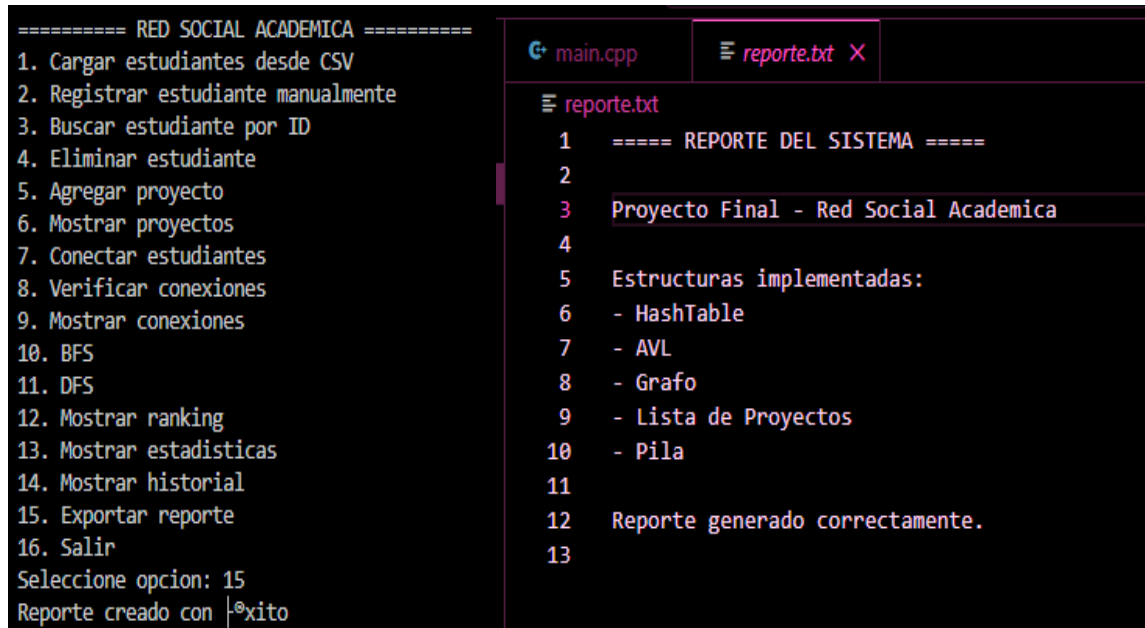
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Run: Benchmark + v [ ] [ ] ... | [ ] [ ] x
PS C:\Users\Daniela Caldera\C++ & java\PROYECTO FINAL - Programación III\Benchmark_java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe'
'--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Daniela Caldera\AppData\Roaming\Code\User\workspaceStorage\33ea
f671ac44ada3f10b50cf3ca0ca5a\redhat.java\jdt_ws\Benchmark_java_ce70e25c\bin' 'Benchmark'
Benchmark Java generado correctamente.
PS C:\Users\Daniela Caldera\C++ & java\PROYECTO FINAL - Programación III\Benchmark_java>
```



language	operation	structure	records	time_ms
Java	insert	HashMap	10000	1.9388
Java	search	HashMap	10000	1.5841
Java	insert	TreeMap	10000	6.0887
Java	insert	TreeMap	10000	6.0887
Java	insert	TreeMap	10000	6.0887

El benchmark desarrollado en Java genera automáticamente el archivo java_results.csv.

MEJORA: Exportación avanzada de reportes



The screenshot shows a code editor with two tabs: 'main.cpp' and 'reporte.txt'. The 'main.cpp' tab is active, displaying a menu for 'RED SOCIAL ACADEMICA' with 16 options. Option 15, 'Exportar reporte', is selected. Below the menu, it says 'Seleccione opcion: 15' and 'Reporte creado con éxito'. The 'reporte.txt' tab is also visible, showing the content of the generated report.

```
===== RED SOCIAL ACADEMICA =====
1. Cargar estudiantes desde CSV
2. Registrar estudiante manualmente
3. Buscar estudiante por ID
4. Eliminar estudiante
5. Agregar proyecto
6. Mostrar proyectos
7. Conectar estudiantes
8. Verificar conexiones
9. Mostrar conexiones
10. BFS
11. DFS
12. Mostrar ranking
13. Mostrar estadísticas
14. Mostrar historial
15. Exportar reporte
16. Salir
Seleccione opcion: 15
Reporte creado con éxito

reporte.txt
1  ===== REPORTE DEL SISTEMA =====
2
3  Proyecto Final - Red Social Academica
4
5  Estructuras implementadas:
6  - HashTable
7  - AVL
8  - Grafo
9  - Lista de Proyectos
10 - Pila
11
12 Reporte generado correctamente.
13
```

3. Explicación de las Estructuras Utilizadas

Tabla Hash

Se utilizó para almacenar estudiantes mediante su identificador único, permitiendo búsquedas rápidas.

Árbol AVL

Se utilizó para mantener un ranking ordenado de estudiantes según su skill score.

Lista Enlazada

Permite almacenar proyectos asociados a los estudiantes.

Pila

Se utiliza para registrar el historial de acciones realizadas dentro del sistema.

Grafo

Representa las conexiones académicas entre estudiantes.

BFS

Permite recorrer la red académica por niveles.

DFS

Permite explorar la red académica en profundidad.

4. Tabla Comparativa de Benchmark

Lenguaje	Operación	Estructura	Registros	Tiempo (ms)
C++	Insert	HashTable	10000	33.391
C++	Search	HashTable	10000	21.428
C++	Insert	AVL	10000	22.226
Java	Insert	HashMap	10000	1.9388
Java	Search	HashMap	10000	1.5841
Java	Insert	TreeMap	10000	6.0887

5. Conclusiones Técnicas

1. Las estructuras de datos influyen directamente en el rendimiento y organización de la información.
2. La Tabla Hash presentó un excelente desempeño en operaciones de búsqueda e inserción.
3. El Árbol AVL permitió mantener los datos ordenados mientras conservaba un rendimiento eficiente.
4. Las estructuras nativas de Java simplifican el desarrollo gracias a las optimizaciones incorporadas por el framework.
5. La implementación manual en C++ permitió comprender con mayor profundidad el funcionamiento interno de cada estructura.